Il Fantasma v0.02: dalla cancellazione dichiarata alla scomparsa misurata

Crypto-shredding, audit scrypt, finestra temporale ed evidenza su GSM8K

dr. Francesco Bulla — Bologna, 10/10/2026

Preprint locale, bozza non pubblicata — v0.02

Sommario

Nella v0.01 il Fantasma "scompariva" per dichiarazione: svuotare due dizionari in Python. Questa nota trasforma la cancellazione in una proprietà misurabile. Definiamo tre livelli di sparizione (L1 store applicativi, L2 memoria del processo, L3 terze parti e provider), sostituiamo l'offuscamento con crypto-shredding in sola stdlib, l'impegno HMAC con scrypt memory-hard, il codice a 13 cifre con 19 cifre canoniche, e aggiungiamo una finestra temporale dichiarata che risolve il paradosso verificabile-versus-riservato invece di aggirarlo. Ogni garanzia è condizionata al proprio report di misura: 44 test verdi, scansione del filesystem, controllo positivo e negativo obbligatori, misura su Linux via Docker. Su 50 problemi GSM8K con DeepSeek-V4.1-Flash il Fantasma ottiene 47/50 contro 46/50 del diretto e del CoT nascosto, con audit verificato e amnesia completa in tutti i giri. Il segnale è debole e lo dichiariamo: è un pilota, non una prova. Il campo per la v0.03 è pronto.

1 Cos'è il Fantasma e perché è diverso

Il Fantasma è uno strato di ragionamento interno che emerge prima della generazione, produce un pensiero in un codice numerico privato, lo fa rileggere dallo stesso modello, riformula la richiesta e poi distrugge la chiave che rende il pensiero leggibile. L'utente vede solo l'output finale; l'audit vede solo impegni, mai contenuti.

Approccio	Pensiero	Cancellazione	Chi può rileggerlo
CoT visibile	nel testo	nessuna	chiunque
Scratchpad nascosto	nascosto all'utente	svuotamento dichiarato	operatore, provider
Latent reasoning	vettori continui	nessuna	chi ha i pesi
Fantasma v0.02	codice a 19 cifre	**distruzione di chiave**	nessuno dopo T

La differenza non è dove il pensiero sta, ma **come muore**: non per dimenticanza, per distruzione del materiale che lo rende leggibile. Il resto — la riformulazione — sopravvive per scelta: è l'utilità del Fantasma, non una perdita.

2 Il problema: la scomparsa era una frase

Misure della revisione sul prototipo v0.01: il codice generato a runtime lascia 3 copie nella memoria del processo, 4 se tenuto in un unico `bytearray`, 10 durante la costruzione del prompt, 8 dopo wipe più garbage collector. Con il codice scritto come letterale nel sorgente, 31 copie: ogni f-string, log, traceback, richiesta HTTP e buffer di socket ne crea una. E lo shift delle cifre della v0.01 è un Cesare a 10 possibilità: si legge a mano. Da qui il threat model, una pagina, tre livelli:

Livello	Promessa	Stato v0.02
L1 — applicativo	nessun riferimento negli store e nei file	garantito e misurato
L2 — processo	nessuna copia inattesa nella RAM ispezionabile	misurato, mai garantito
L3 — provider e rete	nessuna copia da nessuna parte	mai affermato

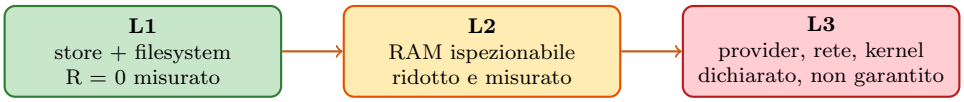


Figura 1: I tre livelli di sparizione. Solo L1 è una garanzia; L2 è una misura; L3 è un limite dichiarato.

Su 43 bit — 13 cifre — verificabile e non-invertibile sono mutuamente esclusivi: se il commitment è verificabile, chiunque può enumerare 10^{13} candidati e aprirlo. La v0.02 prende entrambe le uscite: alza l'entropia a 19 cifre e dichiara la finestra T.

3 Architettura v0.02: cosa cambia davvero

`src/fantasma`: `secret.py` (`SecretBytes` in bytearray wipabile, `OneTimeKey`, `SecretRegion` con wipe per costruzione anche su eccezione), `crypto.py` (encrypt-then-MAC in stdlib su bytearray, scrypt, `TimedCommitment`), `ghost_window.py` (guard rail contro il fork accidentale, rilascio del traceback), `hygiene.py` (mlock puntuale sulla regione, mai mlockall), `forensics.py` (canary runtime, 7 codifiche, scan di store e filesystem, R come copie inattese, claim condizionale), `pipeline_v02.py` (stessa interfaccia `.process()`, interno nuovo), `bench_gsm8k.py` (harness GSM8K).

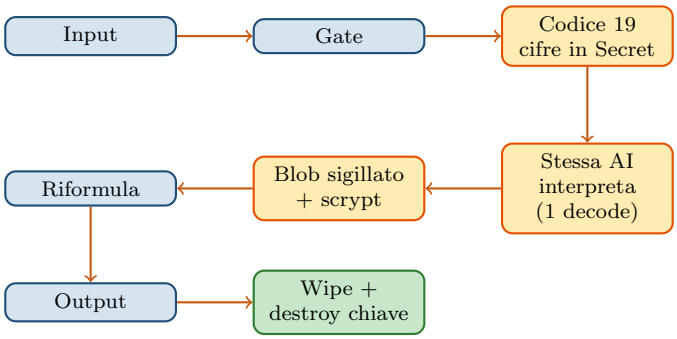


Figura 2: Ciclo v0.02. La cache contiene solo blob indecifrabili; la chiave muore con memset.

Codice canonico a 19 cifre (63,1 bit), mai intero. Lo zero iniziale è significativo; le serializzazioni come numero sono intercettate dalle varianti `int-decimal` e `int-hex` dello scanner.

Scrypt misurato, non sperato. N= 2^{14}, r= 8, p= 1: mediana a regime ∼44 ms per tentativo su un core, ∼14.000 anni/core su 10^{13}. Ma scrypt è bandwidth-bound: su 1 GPU ∼3,2 anni per 13 cifre, su 100 GPU ∼12 giorni. A 19 cifre: milioni di anni anche su GPU. Il numero per-core non è la difesa; la difesa è l'entropia.

Finestra temporale dichiarata. Entro T il commitment è verificabile via tag keyed a chiave viva; dopo `expire()` resta solo scrypt one-way. È la voce "limiti temporali" della Visione, applicata all'audit: verificabile entro T, senso unico dopo T.

Wipe per costruzione. `SecretRegion` fa memset via ctypes in `__exit__` sempre, anche su return anticipata o eccezione. Misurato: riferimento trattenuto più return anticipata danno comunque buffer zerato. `expose_mutable()` resta API di basso livello; la pipeline passa dal manager.

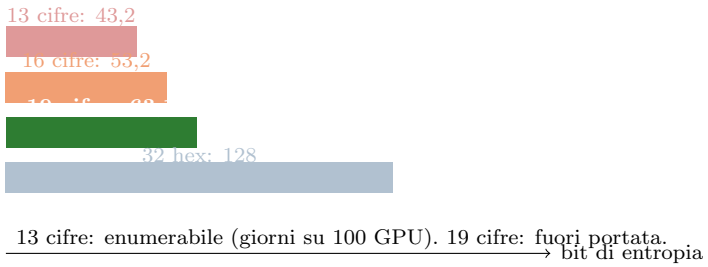


Figura 3: La leva vera è l'entropia, non il KDF. Su 43 bit verificabile e riservato si escludono.

Igiene puntuale. Niente mlockall: solo mlock sulla regione del secret con report per-pagina. Il report dice "regione designata protetta", mai "niente in swap": le copie fuori regione non sono lockate. Il claim L2 pretende lock riuscito più core dump off più scan a zero, altrimenti non è affermato.

4 Claim condizionale: garanzia contro evidenza

Una garanzia è una frase con la guardia davanti. `evaluate_claim()`: L1 affermabile sempre sui propri store; L2 solo con hygiene verde e scan pulito; L3 mai. R = copie trovate meno copie legittime dichiarate: lo scanner possiede il canary per cercarlo, quella copia si sottrae, altrimenti R misura se stesso. Controllo positivo e negativo obbligatori a ogni run: senza positivo che trova la leak iniettata, R=0 non significa niente. "Non trovato" vale solo per le sette codifiche cercate: raw, UTF-16LE/BE, hex, base64, int-decimal, int-hex.

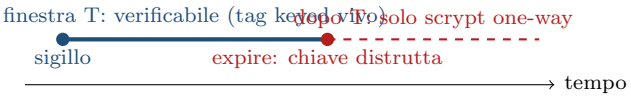


Figura 4: Il paradosso risolto invece che aggirato: verificabile entro T, senso unico dopo T.

5 Limiti misurati, non nascosti

`bytes` non si sovrascrive, `bytearray` sì: la chiave vive in buffer mutabile, il drop del riferimento è igiene, non cancellazione. `del` e `gc` liberano senza azzerare: L1 a zero vale per gli store, non per la RAM. Lo scrub di `f_locals` è write-through solo da Python 3.13 e solo su frame attivi: sui frame sospesi in traceback la scrittura non si propaga (misurato, 0 verificato); la via robusta resta `finally` più rilascio del traceback. `ghost_window` impedisce il fork accidentale da codice Python, non è una barriera contro subprocess o estensioni C; su Windows fork non esiste e lo start method è spawn. AES-GCM via OpenSSL copia il plaintext in C: per il claim di sparizione la stdlib in-place è architetturalmente migliore, e la migrazione ad AEAD resta prevista in v1.0. Su Windows senza `/proc` e senza mlock, L2 è non-misurabile e non affermato: è la guardia che lavora, non un fallimento.

Misura Docker (Linux 6.6 WSL2, Python 3.11.17, hygiene verde, lock su 19 byte), canary a 19 cifre, regioni di memoria contenenti il canary:

6 Evidenza: GSM8K a 50

Harness in `src/fantasma/bench_gsm8k.py`: 50 problemi del test set, seed fisso 20261010, tre bracci sullo stesso DeepSeek-V4.1-Flash con stesso formato finale `#### numero`. Risultati in `outputs/benchmark/`.

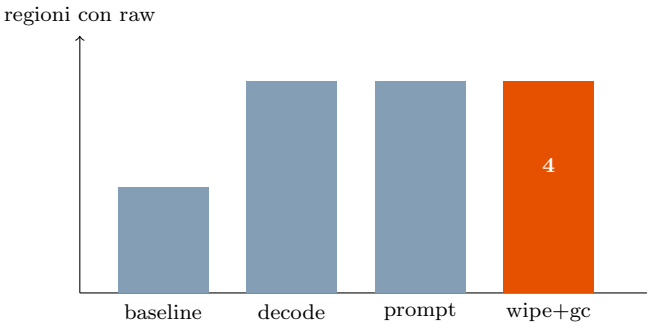


Figura 5: L2 non torna a zero: baseline 2, dopo decode 4, dopo wipe e gc ancora 4. Ridotto e misurato.

Braccio	Accuratezza	Tempo medio	Note
Direct	46/50 = 0.920	2.6 s	una chiamata
CoT nascosto	46/50 = 0.920	2.8 s	una chiamata
Fantasma v0.02	**47/50 = 0.940**	10.7 s	due chiamate + sigillo

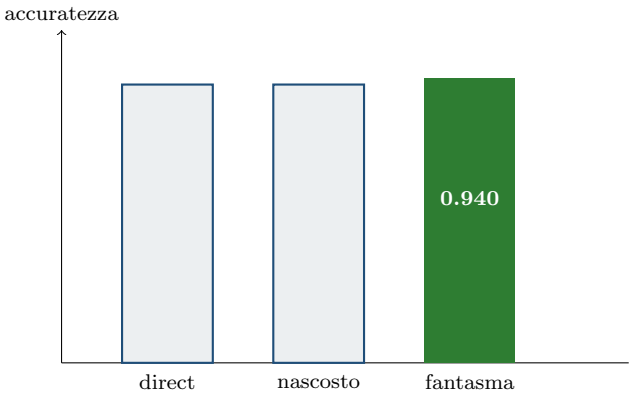


Figura 6: GSM8K a 50: una tacca sopra, a pari modello. Pilota, non prova.

Lato garanzie sul giro fantasma: ghost attivo 48/50 (2 standby legittimi), audit verificato 50/50, memoria e cache vuote 50/50, formato valido sempre. Un solo problema separa i bracci — gold 54, fantasma giusto, gli altri a 27 sbagliati insieme — e nessun caso dove il fantasma sbaglia da solo. Con un modello già al 92% il segnale è debole per costruzione: lo dichiariamo pilota.

7 Cosa cambia dalla v0.01

Cesare → shredding stdlib; HMAC nudo → scrypt; 13 cifre → 19 canoniche; drop del riferimento → memset per costruzione; mlockall → mlock puntuale; “scompare” → L1/L2/L3 con guardie; audit senza finestra → finestra T; 12 test → 44 test; nessun benchmark → GSM8K a 50 con tre bracci.

8 Verso la v0.03: il campo preparato

La domanda resta senza risposta dalla prima nota: il Fantasma migliora davvero le risposte rispetto al CoT nascosto? Il campo è pronto: harness a tre bracci, seed fissi, formato validato. La v0.03 allarga a 200 problemi con campionamento stratificato sui casi dove il diretto sbaglia, aggiunge significatività statistica, prova il modello locale open-weights dove L3 si restringe, e

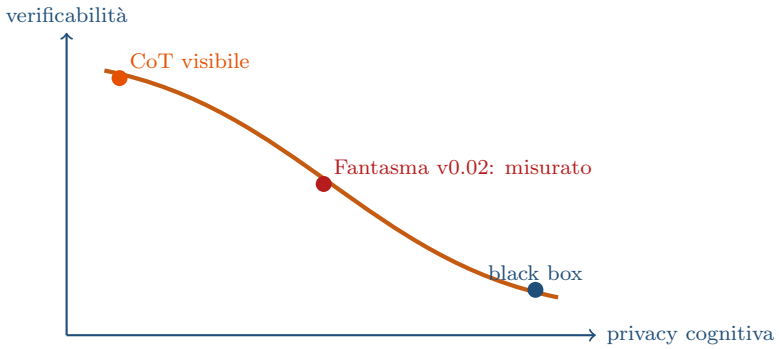


Figura 7: Stessa curva della v0.01, ma il punto del Fantasma ora ha coordinate misurate.

introduce la supervisione indipendente del gate con reset selettivo. Il resto — supervisione, limiti temporali estesi, biforcazione locale/remoto — è la roadmap v1.0.

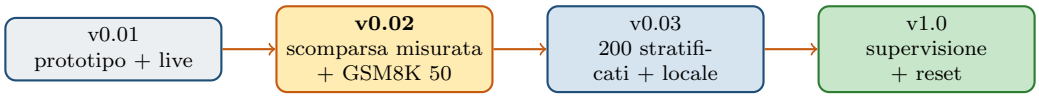


Figura 8: Roadmap aggiornata: la misura c'è, ora la significatività.

9 Stato e riproducibilità

44 test verdi (`pytest tests/`), 12 originali intatti. Misure L2 su Linux 6.6 WSL2, Python 3.11.17, Docker con memlock sbloccato; misure applicative su Windows 10.0.26200, Python 3.13.13. Dataset `openai/gsm8k` split test, seed 20261010, modello `deepseek-flash` con `reasoning_effort=high`. Chiave solo da env, mai nel codice. Altrimenti resta nel quaderno: questa, per ora, misura invece di promettere.